

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 645 — Geometric Modeling

Curves, surfaces, meshes, and the discrete differential geometry that makes them computable

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 2 (with CSCE 611 and CSCE 606)

Prerequisites: CSCE 641 Computer Graphics; linear algebra; multivariable calculus helpful but developed as needed

Effort: 10–12 hours per week · 13 modules + 2 project weeks

Deliverable: A mesh processing library: half-edge structure, curvature, smoothing, parameterisation, simplification, and remeshing — all written by you

Course description

CSCE 641 treated geometry as something that arrives in a vertex buffer. This course is about where that geometry comes from, how it is represented, and what you can compute from it.

The central difficulty is that classical differential geometry is about smooth surfaces, and computers hold triangle meshes. A triangle mesh has zero curvature on every face and infinite curvature on every edge, so the classical definitions do not transfer. **Discrete differential geometry** is the discipline of finding discrete definitions that preserve the theorems rather than the formulas — and it turns out that insisting on the theorems is what produces algorithms that work.

The course runs from the representation question through curves and spline surfaces, into subdivision, and then into the discrete operators — curvature, the Laplacian — that drive almost every modern geometry processing algorithm. Smoothing, parameterisation, simplification, and deformation all turn out to be the same small set of linear systems wearing different clothes.

Everything is implemented. By the end you will have written a mesh library you understand completely, which is a far better foundation for engine work than familiarity with someone else's.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Choose a shape representation from the operations a task requires.
- 2 Implement a half-edge mesh and its traversal operations.

- 3 Evaluate Bézier and B-spline curves and reason about continuity.
- 4 Implement Catmull–Clark and Loop subdivision.
- 5 Compute discrete curvature and explain why the discrete definitions are chosen as they are.
- 6 Build and solve the cotangent Laplacian for smoothing, parameterisation, and deformation.
- 7 Implement quadric-error simplification with attribute preservation.
- 8 Explain implicit representations and convert between them and meshes.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	Representing Shape	What is the right data structure for a shape?
2	Meshes and Connectivity	How do you answer 'what is next to this?' in constant time?
3	Curves: Bézier and Continuity	How do you define a smooth curve from a handful of points?
4	B-Splines and NURBS	How do you build a long smooth curve you can actually edit?
5	Surfaces: Tensor Products and Patches	How do you extend a curve scheme to a surface?
6	Subdivision Surfaces	How do you get a smooth surface on arbitrary topology?
7	Discrete Curvature	What is the curvature of a triangle mesh, which is flat everywhere?
8	The Laplacian and Smoothing	Why does the same matrix solve smoothing, parameterisation, and deformation?
9	Parameterization	How do you lay a curved surface flat with the least damage?
10	Simplification and Remeshing	How do you remove 90% of the triangles and keep the shape?
11	Implicit Surfaces and Distance Fields	What can you do with a shape defined as $f(x) = 0$?
12	Solid Modeling and Booleans	Why are mesh booleans so unreliable?
13	Deformation and Shape Editing	How do you edit a shape while preserving its detail?

Projects

Project 1 — A mesh processing library (after Module 8)

Build the half-edge structure and the operators that everything later depends on. This is infrastructure, and the rest of the course assumes it works.

Requirements

- Half-edge data structure built from an indexed mesh, with detection and reporting of non-manifold input rather than silent failure.
- All one-ring traversals: faces around a vertex, vertices around a vertex, edges of a face, the face across an edge.
- Vertex normals by both uniform and area-weighted averaging, with a comparison.
- Discrete Gaussian and mean curvature, validated against a sphere and a torus where the analytic answer is known.
- The cotangent Laplacian as a sparse matrix, with explicit handling of boundary vertices.
- Explicit and implicit Laplacian smoothing.

Done means

- Gaussian curvature integrated over a closed genus-0 mesh equals 4π to within floating-point tolerance. Report the number.
- The same integral over a torus equals 0. Report it.
- A noisy scanned mesh smoothed with both explicit and implicit schemes, with the time step at which explicit smoothing becomes unstable reported.

Project 2 — Simplify, parameterise, remesh (after Module 12)

Take a scanned model through a complete processing pipeline and produce something an engine could actually load.

Requirements

- Quadric-error simplification with UV and normal seams preserved, producing a clean LOD chain.
- Least-squares conformal parameterisation of a mesh cut into charts, with the distortion measured and reported.
- Isotropic remeshing to a target edge length.
- A normal map baked from the high-resolution mesh onto the simplified one.
- Everything rendered in your CSCE 641 renderer.

Done means

- An LOD chain at 100%, 25%, and 5% of the original triangle count, shown side by side at matched screen size.
- A UV atlas image with distortion visualised as a per-triangle colour.
- The simplified mesh with the baked normal map rendered beside the original, at a distance where they should be indistinguishable.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Keenan Crane — Discrete Differential Geometry (free notes, video, and code)	Primary source. Complete lecture videos, a full set of course notes, and C++/JavaScript skeleton code. The best free resource on this subject by a wide margin.
libigl tutorial (free, with code)	A working geometry processing library with a tutorial that doubles as a survey of the field. Read the source for every algorithm you implement.
Polygon Mesh Processing — course slides (free)	Slides and the companion open-source library. Strong on simplification, remeshing, and parameterisation.
GAMES101 Lectures 11–12 — Geometry	The condensed version you met in CSCE 641. Useful as revision before Module 03.
MIT 6.8410 Shape Analysis — Justin Solomon	A graduate course covering the same operators from a more analytical direction. Good second voice for Modules 07–09.
Inigo Quilez — articles on distance functions (free)	The practical reference for implicit surfaces and SDFs, with derivations and working shader code. Primary source for Module 11.

Tooling

- **C++17** with **Eigen** for linear algebra. You will be solving sparse linear systems from Module 08 onward and writing your own solver is not the point of this course.
- **libigl** as a reference implementation and for mesh I/O. Write your own algorithms; use libigl to check them.
- **Polyscope** for visualisation — a few lines to display a mesh with per-vertex scalar fields, which you will want constantly.
- **A test mesh collection**: a sphere and torus (known curvature), a cube (sharp features), a scanned model (noise and holes), and a deliberately broken mesh (non-manifold edges).
- **A portfolio repository** with a rendered image for every algorithm.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.